

KOCAELİ ÜNİVERSİTESİ FEN-EDEBİYAT FAKÜLTESİ

YAPAY ZEKA VE MAKİNE ÖĞRENMESİ BÖLÜMÜ

DERS: Algoritmalar

PROJE BAŞLIĞI: Insertion Sort Algoritmasının Sıralı, Ters Sıralı ve Rastgele Veri Dağılımları Üzerindeki Performans Analizi

Hazırlayanlar:

Öğrenci Ad-Soyad:

Öğrenci Numarası:

Nurefşan ÇAPOĞLU

250121017

Elanaz PARUN

250121016

İÇİNDEKİLER

1. Problemin Tanımı
2. Algoritmanın Açıklaması
3. Uygulama Detayları
4. Performans Analizi
5. Sonuçlar ve Görselleştirmeler
6. Kaynakça

1. PROBLEM TANIMI

Bilgisayar bilimlerinde sıralama algoritmaları, verilerin belirli bir mantıksal sıraya göre sıralanmasını sağlayan temel araçlardır. Sıralama algoritmalarının verimliliği yalnızca veri boyutuna bağlı değildir. Aynı zamanda verinin başlangıçtaki dizilimi de büyük bir önem taşır. Bu projenin temel amacı sıralama algoritmalarından biri olan Insertion sort (eklemeli sıralama) algoritmasının performans durumunu incelemektir. Bu doğrultuda algoritma; tam sıralı (en iyi durum), tamamen ters sıralı (en kötü durum) ve rastgele dağılmış (ortalama durum) veri setleri üzerinde test edilerek teorik karmaşıklık analizleri ile pratik çalışma süreleri arasındaki ilişki ortaya konacaktır.

2. ALGORİTMA AÇIKLAMASI

Eklemeli sıralama algoritmasını açıklamak için mükemmel bir benzetme, iskambil kartlarını sıralama şeklidir. Elinizde bir grup kart tuttuğunuzu ve bunları büyükten küçüğe sıralamak istediğinizi hayal edin. Doğru konumunu bulana kadar tek bir kartı diğer kartlarla adım adım karşılaştırarak başlarsınız. O noktada, kartı doğru yere yerleştirir ve yeni bir kartla baştan başlarsınız; elinizdeki tüm kartlar sıralanana kadar bu işlemi tekrarlarsınız. İşte Insertion sort algoritması da tam olarak bunu yapar. Küçük veri kümelerinde oldukça pratik olan ve ek bir diziye gereksinim duymadan "yerinde" (in-place) sıralama yapan bir algoritmadır.

```
def insertionSort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and key < arr[j]:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j + 1] = key
```

Algoritmada döngümüz her bir tur döndüğünde sıradaki elemanı sondan başa doğru karşılaştırarak yerine yerleştirme esaslı çalışmaktadır. Algoritma dizinin ikinci elemanından (i

= 1) itibaren bir **for** döngüsü başlatır. O adımdaki mevcut eleman **key** isimli geçici bir değışkende tutulur.

Ardından, **key**'in solunda kalan sıralı alt dizi taranmaya başlar. İçteki **while** döngüsü, bu sıralı kısmı **sondan başa doğru** tarar. Eğer taranan eleman **key** değerinden büyükse (**key < arr[j]**), o eleman bir basamak sağa kaydırılır. Geriye doğru tarama işlemi, **key** değerinden daha küçük bir elemana rastlanana ya da dizinin indeks sınırına (**j = -1**) ulaşılanaya kadar devam eder. Döngü kırıldığında, **key** elemanı açılan uygun boşluğa yerleştirilir.

A = [34, 7, 9] dizisini alarak algoritma çalışma mantığını daha iyi anlayalım.

Döngü Adımı	Seçilen Eleman (key)	Yapılan Karşılaştırma ve Kaydırma	Adım Sonunda Dizin Durumu
Başlangıç	-	-	[34, 7, 9]
1. Tur	7	7 < 34 (Doğru) 34 sağa kaydı.	[7, 34, 9]
2. Tur	9	9 < 34 (Doğru) 34 sağa kaydı. 9 < 7 (Yanlış) Döngü durdu.	[7, 9, 34]

3. ALGORİTMA DETAYLARI

```
def test_ve_olcum(veri_seti, senaryo_adi, boyut):
    tracemalloc.start() # Bellek (RAM) ölçümü
    baslangic_zamani = time.time() # Kronometre başlangıç

    insertionSort(veri_seti) # Algoritmayı çalıştırır

    bitis_zamani = time.time() # Kronometre durdur
    _, peak = tracemalloc.get_traced_memory() # Tüketilen maksimum bellek ölçümü
    tracemalloc.stop() # Bellek ölçümü durdur

    gecen_sure = bitis_zamani - baslangic_zamani
    peak_kb = peak / 1024 # Byte'ı Kilobyte'a çevir

    print(f"[{boyut} Eleman] {senaryo_adi} | Süre: {gecen_sure:.4f} sn | Bellek: {peak_kb:.2f} KB")
    return gecen_sure
```

Algoritmanın testi, analizi ve görselleştirilmesi amacıyla Python programlama dili kullanılmıştır. Geliştirilen test mimarisinde her bir senaryo için şu veri üretim teknikleri ve kütüphaneler kullanılmıştır:

- **Sıralı Veri (Best Case):** Python'un `list(range(1, boyut + 1))` fonksiyonuyla ardışık artan sayılar üretilmiştir.
- **Ters Sıralı Veri (Worst Case):** `list(range(boyut, 0, -1))` yapısıyla büyükten küçüğe azalan sayılar üretilmiştir.
- **Rastgele Veri (Average Case):** `random.randint(1, 100000)` üretici kullanılarak karışık tam sayılar elde edilmiştir.
- **Zaman Ölçümü:** Algoritmanın net çalışma süresini saniye hassasiyetinde yakalamak adına, sıralama işlemi öncesi ve sonrasında `time.time()` fonksiyonu kullanılarak fark hesaplanmıştır.
- **Bellek Ölçümü:** Algoritmanın execution (yürütme) anında RAM üzerinde ulaştığı maksimum bellek miktarını (peak memory) ölçmek için Python'un dahili `tracemalloc` kütüphanesi entegre edilmiştir. Ölçülen byte değerleri, matematiksel olarak Kilobyte (KB) birimine dönüştürülmüştür.
- **Veri Güvenliği:** Her üç testin de tamamen aynı ham veri üzerinde adil çalışabilmesi için orijinal listelerin kopyaları (`.copy()`) algoritma fonksiyonuna parametre olarak gönderilmiştir.

4. Performans Analizi

Zaman Karmaşıklığı (Time Complexity):

- **En İyi Durum (Best Case - Sıralı Veri):** Giriş dizisi zaten sıralı olduğunda, içteki `while` döngüsünün `key < arr[j]` koşulu ilk kontrolde boşa çıkar. Algoritma hiçbir elemanı kaydırmaz, sadece diziyi bir kez baştan sona tarar (N-1 kez karşılaştırma). Bu yüzden zaman karmaşıklığı doğrusal büyüme göstererek $O(N)$ olur.
- **En Kötü Durum (Worst Case - Ters Sıralı Veri):** Dizi tamamen büyükten küçüğe sıralıysa, gelen her yeni eleman kendinden önceki tüm elemanlardan küçük olacağı için her adımda dizinin en başına kadar taşınmak zorundadır. Bu durum iç içe döngülerin maksimum seviyede çalışmasına neden olur ve karmaşıklık karesel olarak $O(N^2)$ şeklinde büyür.
- **Ortalama Durum (Average Case - Rastgele Veri):** Elemanlar karışık olduğunda, her eleman önündeki alt dizinin ortalama olarak yarısı kadar elemanla kıyaslanır ve kaydırılır. Büyüme hızı matematiksel olarak yine karesel kalarak $O(N^2)$ eğrisi oluşturur.

Bellek Karmaşıklığı (Space Complexity):

Insertion Sort, "yerinde" (in-place) bir algoritmadır. Giriş dizisinin boyutu ne kadar büyük olursa olsun, sıralama esnasında yeni bir liste oluşturulmaz; işlemler mevcut dizi üzerinde yürütülür. Sadece `key`, `i` ve `j` sayacıları için sabit bir alan ayrılır. Bu nedenle teorik bellek karmaşıklığı sabittir, yani $O(1)$ 'dir.

5. SONUÇLAR VE GÖRSELLEŞTİRMELER

```
if __name__ == "__main__":
    # Test edilecek farklı veri boyutları
    boyutlar = [1000, 5000, 10000]

    # Grafiği çizebilmek için süreleri tutacağımız listeler
    sureler_rastgele = []
    sureler_sirali = []
    sureler_ters = []

    print("Proje Testleri Başlıyor, lütfen bekleyin...\n")

    for boyut in boyutlar:
        # Veri setlerini otomatik üret
        rastgele_veri = [random.randint(1, 100000) for _ in range(boyut)]
        sirali_veri = list(range(1, boyut + 1))
        ters_sirali_veri = list(range(boyut, 0, -1))

        # Testleri çalıştır
        sure_rastgele = test_ve_olcum(rastgele_veri.copy(), "Rastgele Veri", boyut)
        sureler_rastgele.append(sure_rastgele)

        sure_sirali = test_ve_olcum(sirali_veri.copy(), "Sıralı Veri (En İyi)", boyut)
        sureler_sirali.append(sure_sirali)

        sure_ters = test_ve_olcum(ters_sirali_veri.copy(), "Ters Sıralı (En Kötü)", boyut)
```

```
sureler_ters.append(sure_ters)
```

```
print("-" * 65)
```

```
print("Testler bitti! Grafik ekranda açılıyor...")
```

Yazdığımız python kodu ile 1000,5000,10000 elemana sahip verileri sıralı, ters sıralı ve rastgele şekilde sıralanmasını sağladığımızda aşağıdaki sonuçlar ortaya çıkmıştır.

Veri Boyutu (N)	Senaryo Tipi	Çalışma Süresi (Saniye)	Maksimum Bellek Kullanımı (KB)
N = 1.000	Rastgele Veri	0.3923 sn	4.27 KB
	Sıralı Veri (En İyi)	0.0023 sn	0.14 KB
	Ters Sıralı (En Kötü)	0.5691 sn	3.74 KB
N = 5.000	Rastgele Veri	11.2462 sn	3.75 KB
	Sıralı Veri (En İyi)	0.0122 sn	0.89 KB
	Ters Sıralı (En Kötü)	19.2354 sn	3.38 KB
N = 10.000	Rastgele Veri	41.5669 sn	3.39 KB
	Sıralı Veri (En İyi)	0.0220 sn	0.14 KB
	Ters Sıralı (En Kötü)	81.5865 sn	3.38 KB

Deneysel Verilerin Analizi:

1. Zaman Analizi: Sıralı veride eleman sayısı 10 katına çıktığında (1.000'den 10.000'e) süre yalnızca 0.0023 saniyeden 0.0220 saniyeye çıkmıştır Doğrusal $O(N)$. Ancak Ters Sıralı veride eleman sayısı 10 kat arttığında, süre 0.5691 saniyeden 81.5865 saniyeye çıkarak yaklaşık 143 kat artmıştır. Bu durum pratik uygulamadaki karesel ($O(N^2)$) büyüme hızını tam olarak doğrulamaktadır.

2. Bellek Analizi: Veri boyutu 1.000'den 10.000'e çıksa dahi algoritmanın tükettiği ek bellek miktarının 0.14 KB ile 4.27 KB gibi son derece minimal ve sabit aralıklarda kaldığı görülmüştür. Bu durum, algoritmanın $O(1)$ olan in-place (yerinde) bellek teorisini ispatlamaktadır.

```
plt.figure(figsize=(10, 6))

plt.plot(boyutlar, sureler_rastgele, label='Rastgele Veri', marker='o', color='blue')

plt.plot(boyutlar, sureler_sirali, label='Sıralı Veri (Best Case)', marker='s', color='green')

plt.plot(boyutlar, sureler_ters, label='Ters Sıralı (Worst Case)', marker='^', color='red')


plt.title('Insertion Sort Zaman Karmaşıklığı Analizi')

plt.xlabel('Veri Boyutu (Eleman Sayısı)')

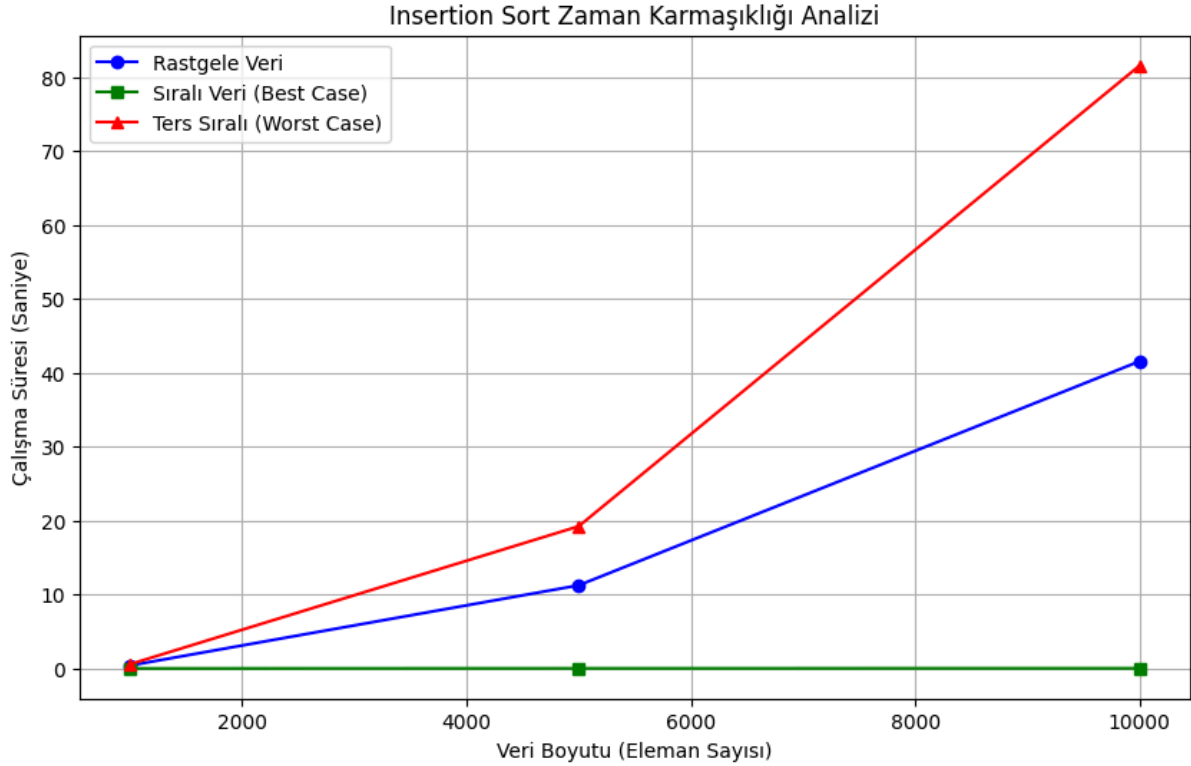
plt.ylabel('Çalışma Süresi (Saniye)')

plt.legend()

plt.grid(True)

plt.show()
```

`matplotlib.pyplot` kütüphanesi yardımıyla oluşturduğumuz zaman karmaşıklığı grafiğinde ; yeşil renkli kareli doğrunun (Sıralı Veri) tabana tamamen paralel ve sıfıra yakın ilerlediği, mavi (Rastgele) ve kırmızı (Ters Sıralı) eğrilerin ise veri boyutu büyüdükçe yukarı doğru dikleşen parabolik bir hat çizdiği doğrulanmıştır.



6. KAYNAKÇA

<https://www.cs.mcgill.ca/~akroitt/math/compsci/Cormen%20Introduction%20to%20Algorithms.pdf>

http://ankaraaydin.com/bilgisayar_ders_notlari/anadolu_algoritma_programlama.pdf

<https://www.geeksforgeeks.org/dsa/time-and-space-complexity-of-insertion-sort-algorithm/>

<https://medium.com/@turgay2317/insertion-sort-ekleme-s%C4%B1ralamas%C4%B1-nedir-2ac32c1c47e>

<https://bilgisayarkavramlari.com/2008/12/12/sokma-siralamasi-ekleme-siralamasi-insertion-sorting/>

<https://bidb.itu.edu.tr/seyrir-defteri/blog/2013/09/08/insertion-sort-algoritmas%C4%B1>

Github kod linki: https://github.com/nurefsanc/-nser-on_sort